
UNIT 10 TRANSACTIONS AND CONCURRENCY MANAGEMENT

Structure

Page Nos.

- 10.0 Introduction
- 10.1 Objectives
- 10.2 The Transactions
 - 10.2.1 Properties of a Transaction
 - 10.2.2 States of a Transaction
- 10.3 Concurrent Transactions
 - 10.3.1 Transaction Schedule
 - 10.3.2 Problems of Concurrent Transactions
- 10.4 The Locking Protocol
 - 10.4.1 Serialisable Schedule
 - 10.4.2 Locks
 - 10.4.3 Two-Phase Locking (2PL)
- 10.5 Deadlock Handling and its Prevention
- 10.6 Optimistic Concurrency Control
- 10.7 Timestamp-Based Protocols
 - 10.7.1 Timestamp Based Concurrency Control
 - 10.7.2 Multi-version Technique
- 10.8 Weak Level of Consistency and SQL commands for Transactions
- 10.9 Summary
- 10.10 Solutions/ Answers

10.0 INTRODUCTION

One of the main advantages of storing data in an integrated repository or a database is to allow sharing of data amongst multiple users. Several users access the database or perform transactions at the same time. What if a user's transactions try to access a data item that is being used /modified by another transaction? This unit attempts to provide details on how concurrent transactions are executed under the control of DBMS. However, in order to explain the concurrent transactions, first this unit describes the term transaction. Concurrent execution of user programs is essential for better performance of DBMS, as concurrent running of several user programs keeps utilising CPU time efficiently, since disk accesses are frequent and are relatively slow in case of DBMS. This unit not only explains the issues of concurrent transaction but also explains algorithms to control those problems.

10.1 OBJECTIVES

After going through this unit, you should be able to:

- define the term database transactions and their properties;
- describe the issues of concurrent transactions;
- explain the mechanism to prevent issues arising due to concurrently executing transactions;
- describe the principles of locking and serialisability; and
- describe concepts of deadlock and its prevention.

10.2 THE TRANSACTIONS

A transaction is a unit of data processing. Database systems that deal with a large number of concurrent transactions are also called Transaction Processing Systems. A

transaction is a unit of work in a database system. For example, a database system for a bank may allow the transactions such as – withdrawal of money from an account; deposit of money to an account; transfer of money from A's account to B's account. A transaction would involve the manipulation of one or more data values in a database. Thus, it may require reading and writing of database values. The following are examples of transactions.

Example 1: A money withdrawal transaction of a bank can be written in pseudo-code as:

; Assume that you are doing this transaction for account number X.

```
TRANSACTION WITHDRAWAL (withdrawal_amount)
Begin transaction
    IF X exists then
        READ X.balance
        IF X.balance > withdrawal_amount
            THEN
                SUBTRACT withdrawal_amount from X.balance
                WRITE X.balance
                Dispense withdrawal_amount
                COMMIT
            ELSE
                DISPLAY "TRANSACTION CANNOT BE PROCESSED"
        ELSE DISPLAY "ACCOUNT X DOES NOT EXIST"
    End transaction.
```

Another similar example may be the transfer of money from account number X to account number Y. This transaction may be written as:

Example 2: Transaction to transfers some amount from account X to account Y.

```
TRANSACTION (X, Y, transfer_amount)
Begin transaction
    IF X AND Y exist then
        READ X.balance
        IF X.balance > transfer_amount THEN
            X.balance = X.balance - transfer_amount
            READ y.balance
            Y.balance = Y.balance + transfer_amount
            COMMIT
        ELSE DISPLAY ("INSUFFICIENT BALANCE IN X")
            ROLLBACK
    ELSE DISPLAY ("ACCOUNT X OR Y DOES NOT EXIST")
    End transaction
```

Please note the following:

The two keywords here COMMIT and ROLLBACK are:

COMMIT makes sure that all the changes made by transactions are made permanent.

ROLLBACK terminates the transactions and rejects any change made by the transaction.

In general, databases are stored in secondary storage as data blocks, whereas they can be processed in the main memory. The portion of main memory allotted for database processing is called a buffer. When a transaction desires to update a value X, a transaction program performs the following operations:

Read the block containing value X to memory buffer

Process the value of X in the memory buffer

Write the value of X to the data block

A block of data on secondary storage may contain many data items, as its size may be in MBs. Further, a transaction processing system may modify several data items simultaneously, therefore, an interesting question for a DBMS is: when to write a data block back to secondary storage? This process is called Buffer management. You may refer to further readings for details on this topic.

Transactions have certain desirable properties. Let us look into the properties of a transaction.

10.2.1 Properties of a Transaction

A transaction has four basic properties. These are:

- Atomicity
- Consistency
- Isolation
- Durability

These are also called the ACID properties of transactions.

Atomicity: It defines a transaction to be a single unit of processing. In other words, either a transaction will be done *completely* or *not at all*. For example, in the transaction of Example 2, the transaction is reading and writing more than one data items. The atomicity property requires either operations on both the data item to be performed or not at all.

Consistency: This property ensures that complete transaction execution takes a database from one consistent state to another consistent state. If a transaction fails even then the database should come back to a consistent state, i.e., either to the database state that was before the start of the transaction or the database state after the end of the transaction.

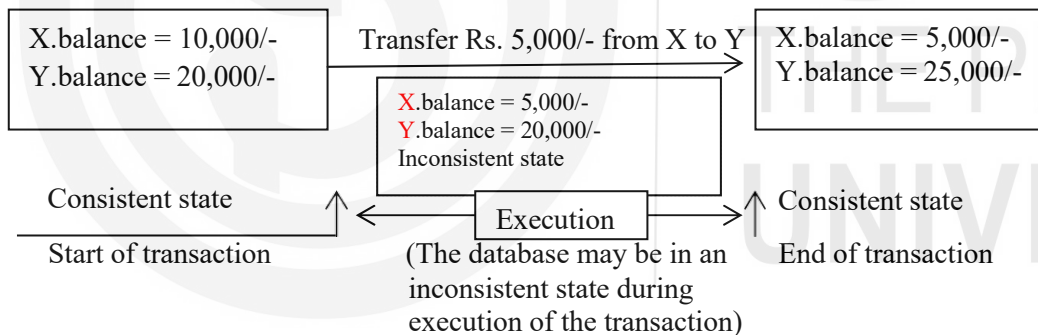


Figure 1: A Transaction execution

Isolation or Independence: The isolation property states that the updates of a transaction should not be visible till they are committed. Isolation guarantees that the progress of a transaction does not affect the outcome of another transaction. For example, if another transaction that is a withdrawal transaction which withdraws an amount of Rs. 5000 from X account is in progress, then failure or success of the transaction of example 2, should not affect the outcome of this transaction. Only the state of the data that has been read by the transaction should determine the outcome of this transaction.

Durability: This property necessitates that once a transaction has been committed, the changes made by it be never lost because of subsequent failure. Thus, a transaction is also a basic unit of recovery. The details of transaction-based recovery are discussed in the next unit.

10.2.2 States of a Transaction

A transaction can be in any one of the states during its execution. These states are displayed in *Figure 2*.

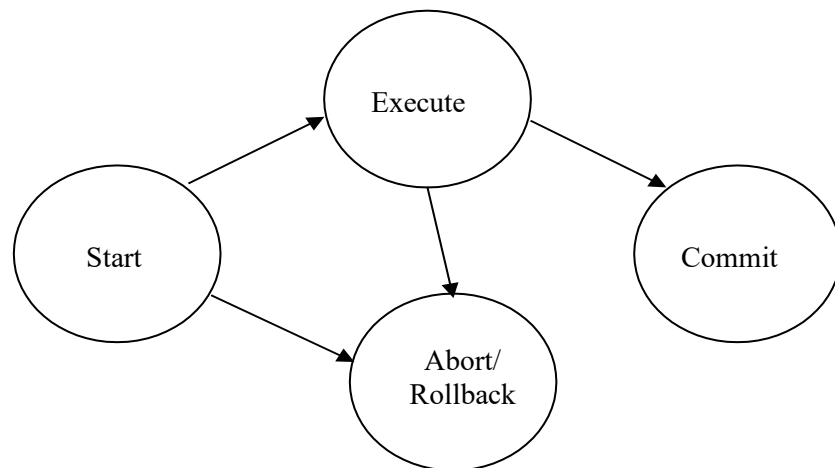


Figure 2: States of transaction execution

A transaction is started by a program. When a transaction is scheduled for execution by the Processor, it moves to the Execute state; however, in case of any system error at that point, it may be moved into the Abort state. During its execution, a transaction changes the data values, which may move the database to an inconsistent state. On successful completion of a transaction, it moves to the Commit state, where the durability feature of transaction ensures that the changes will not be lost. However, in case of an error in the execute state, the transaction goes to the Rollback state, where all the changes made by the transaction are undone. Thus, after commit or rollback, the database is back into consistent state. In case a transaction has been rolled back, it can be started as a new transaction. All these states of the transaction are shown in *Figure 2*.

10.3 THE CONCURRENT TRANSACTIONS

Almost all commercial DBMSs support multi-user environment, allowing multiple transactions to proceed simultaneously. The DBMS must ensure that two or more transactions do not get into each other's way, i.e., a transaction of one user does not affect the transactions of other users or even the other transactions issued by the same user. Please note that concurrency related problems may occur in databases only if **two transactions are contending for the same data item and at least one of the concurrent transactions wishes to update a data value in the database**. In case the concurrent transactions only read the same data item and no updates are performed on these values, then they do NOT cause any concurrency related problem. Now, let us first discuss why you need a mechanism to control concurrent transactions. This is explained next.

10.3.1 Transaction Schedule

Consider a banking application dealing with checking and saving accounts. A Banking Transaction T1 for Mr. Sharma moves Rs.100 from his checking account balance X to his savings account balance Y, using the transaction T1:

Transaction T1:

```
A:Read X
  Subtract
```

```

100
Write X
B:Read Y
Add 100
Write Y

```

Let us suppose an auditor wants to know the total assets of Mr. Sharma. S/he executes the following transaction:

Transaction T2:

```

Read X
Read Y
Display X+Y

```

Suppose both of these transactions are issued simultaneously, then the execution of these instructions can be mixed in many ways. This is also called the **Schedule**. Let us define this term in more detail.

Consider that a database system has n active transactions, namely $T1, T2, \dots, Tn$. Each of these transactions can be represented using a transaction program consisting of operations, as shown in Example 1 and Example 2. A schedule, say S , is defined as the sequential ordering of the operations of the ' n ' interleaved transactions, where the sequence or order of operations of an individual transaction is maintained.

Conflicting Operations in Schedule: Two operations of different transactions conflict if they access the same data item AND one of them is a write operation.

For example, the two transactions TA and TB, as given below, if executed in parallel, may produce a schedule:

TA
READ X
WRITE X

TB
READ X
WRITE X

(a) The two transactions

SCHEDULE	TA	TB
READ X	READ X	
READ X		READ X
WRITE X		WRITE X
WRITE X	WRITE X	

(b) One possible conflicting schedule for interleaved execution of TA and TB

Figure 3: A conflicting schedule

Let us show you three simple ways of interleaved instruction execution of transactions T1 and T2. Please note that in the following tables the first column defines the sequence of instructions that are getting executed, that is the schedule of operations.

Schedule	Transaction T1	Transaction T2	Example Values
Read X		Read X	X = 50000
Read Y		Read Y	Y = 100000
Display X+Y		Display X+Y	150000
Read X	Read X		X = 50000
Subtract 100	Subtract 100		= 49900
Write X	Write X		X = 49900

Read Y	Read Y		Y= 100000
Add 100	Add 100		100100
Write Y	Write Y		Y= 100100

- a) Complete execution of T2 is before T1 starts, then sum X+Y will show the correct assets.

Schedule	Transaction T1	Transaction T2	Example Values
Read X	Read X		X = 50000
Subtract 100	Subtract 100		49900
Write X	Write X		X = 49900
Read Y	Read Y		Y= 100000
Add 100	Add 100		100100
Write Y	Write Y		Y= 100100
Read X		Read X	X = 49900
Read Y		Read Y	Y= 100100
Display X+Y		Display X+Y	150000

- b) Complete execution of T1 is before T2 starts, then sum X+Y will still show the correct assets.

Schedule	Transaction T1	Transaction T2	Example Values
Read X	Read X		X = 50000
Subtract 100	Subtract 100		49900
Write X	Write X		X = 49900
Read X		Read X	X = 49900
Read Y		Read Y	Y= 100000
Display X+Y		Display X+Y	149900
Read Y	Read Y		Y= 100000
Add 100	Add 100		100100
Write Y	Write Y		Y= 100100

- c) Part execution of transaction T1, followed by the complete execution of T2, followed by the remaining part of T1.

Figure 4: Three different possibilities of interleaved execution of T1 and T2

In this execution, an incorrect value is displayed. This is because Rs.100, although removed from X, has not been put in Y and is thus missing from the computation of T2. Please note that for the given transaction there are many more ways of interleaved instruction execution.

Thus, there can be a possibility of anomalies when the transactions T1 and T2 are allowed to execute in parallel. Let us define the anomalies of concurrent execution of transactions more precisely.

10.3.2 Anomalies of Concurrent Transactions

Let us assume the following transactions (assuming there will not be errors in datawhile execution of transactions)

Transaction T3 and T4: T3 reads the balance of account X and subtracts a withdrawal amount of Rs. 5000, whereas T4 reads the balance of account X and adds an amount of Rs. 3000

T3
READ X
SUB 5000
WRITE X

T4
READ X
ADD 3000
WRITE X

The possible problems in the concurrent execution of these transactions are:

1. **Lost Update Anomaly:** Suppose the two transactions T3 and T4 run concurrently, and they happen to be interleaved in the following way (assume the initial value of X as 10000):

T3	T4	Value of X	
		T3	T4
READ X		10000	
	READ X		10000
SUB 5000		5000	
	ADD 3000		13000
WRITE X		5000	
	WRITE X		13000

After the execution of both transactions, the value X is 13000, while the semantically correct value should be 8000. The problem occurred as the update made by T3 has been overwritten by T4. The root cause of the problem was the fact that both the transactions had read the value of X as 10000. Thus, one of the two updates has been lost and we say that a **lost update** has occurred.

There is one more way in which the lost updates can arise. Consider the following part of some transactions T5 and T6; each of which increases the value of X by 1000.

T5	T6	Value of x originally 2000	
		T5	T6
Update X		3000	
	Update X		4000
ROLLBACK		2000	

Here, transactions T5 and T6 update the same item X (please note that an Update include Reading, Modifying and Write operations). Thereafter, T5 decides to undo its action and rolls back, causing the value of X to go back to the original value 2000. In this case, the update performed by T6 had got lost and a lost update is said to have occurred.

2. **Unrepeatable read Anomaly:** Suppose transaction T7 reads X twice during its execution. If it did not update X itself, it could be very disturbing to see a different value of X in its next read. But this could occur if, between the two read operations, another transaction modifies X.

T7	T8	Assumed value of X=2000	
		T7	T8
READ X		2000	
	Update X		3000
READ X		3000	

Thus, inconsistent values are read, and the results of the transaction may be in

error.

3. **Dirty Read Anomaly:** T10 reads a value which has been updated by T9. This update has not been committed and T9 aborts.

T9	T10	Value of x old value =200	
		T9	T10
Update X		500	
	READ X		500
ROLLBACK		200	?

Here T10 reads a value that has been updated by transaction T9 that has been aborted. Thus, T10 has read a value that would never exist in the database and hence the problem.

Please note that all three problems that we have discussed so far are primarily due to the violation of the Isolation property of the concurrently executing transactions that use the same data items.

In addition to the three anomalies, sometimes concurrent transactions may result in inconsistent analysis. For example, in the schedule of the two transactions T1 and T2, as shown in Figure 4(c), you may observe that one part of transaction T1 is executed prior to the execution of transaction T2 and the other part after the completion of Transaction T2. You may also note that transaction T2 produces an incorrect sum of the balance of accounts X and Y. The cause of the problem is that T2 transaction has accessed data items, which are still being modified by another transaction. The important thing to note is that the modifying transaction has been able to modify only some of the data values and some data values are yet to be modified. This resulted in a situation where the analysis transaction has read some modified values and some unmodified values, resulting in an inconsistent output.

As you can observe from the problems discussed in this section, concurrency-related problems occur when multiple transactions contend for a data item concurrently. But how do we ensure that the execution of two or more transactions does not result in concurrency-related problems?

Well, one of the commonest techniques used for this purpose is to restrict access to data items that are being read or written by one transaction and are also being written by another transaction. This technique is called locking. Let us discuss locking in more detail in the next section.

Check Your Progress 1

- 1) What is a transaction? What are its properties? Can a transaction update more than one data value? Can a transaction write a value without reading it? Give an example of a transaction.

.....

- 2) What are the anomalies of concurrent transactions? Can these problems occur in transactions which do not read the same data values?

.....

3) What is a Commit state? Can you rollback after the transaction commits?

10.4 THE LOCKING PROTOCOL

To control concurrency related problems, we use locking. A lock is basically a variable that is associated with a data item in the database. A lock can be placed by a transaction on a shared resource. A locked data item is available for the exclusive use of the transaction that has locked it. Other transactions are locked out of that data item. When a transaction that has locked a data item does not desire to use it anymore, it should unlock the data item so that other transactions can use it. If a transaction tries to lock a data item already locked by some other transaction, it cannot do so and waits for the data item to be unlocked. The component of DBMS that controls and manages the locking and unlocking of data items is called the Lock Manager. The locking mechanism helps us to convert a schedule into a serialisable schedule. We had defined what a schedule is, but what is a serialisable schedule? Let us discuss it in more detail in the next section.

10.4.1 Serialisable Schedule

If the operations of two transactions conflict with each other, how to determine that no concurrency-related problems have occurred in the transaction execution? For this purpose, let us define the term – Schedule, Serial Schedule, interleaved schedule and Serializable Schedule.

A schedule can be defined as a sequence of actions/operations of one or more transactions, as explained in section 10.3.1. Figure 5 shows two schedules, viz. Schedule A and Schedule B.

A serial schedule is one in which the actions/operations of one transaction are performed at a time. This is followed by the actions/operations of the next transaction and so on. For example, Schedule A and Schedule B of Figure 5 are serial schedules, as in no schedule operations of transactions T1 and Transaction T2 interleave with each other.

An interleaved schedule allows the interleaving of actions/operations of different transactions. For example, the schedule in Figure 6 is an interleaved schedule. The basic question here is: How will you find out that a given interleaved schedule has not resulted in concurrency-related problems?

Schedule A: T2 followed by T1			Schedule B: T1 followed by T2		
Schedule A	T1	T2	Schedule B	T1	T2
READ X		READ X	READ X	READ X	
READ Y		READ Y	SUBTRACT 100	SUBTRACT 100	
DISPLAY X+Y		DISPLAY X+Y	WRITE X	WRITE X	
READ X	READ X		READ Y	READ Y	
SUBTRACT 100	SUBTRACT 100		ADD 100	ADD 100	
WRITE X	WRITE X		WRITE Y	WRITE Y	
READ Y	READ Y		READ X		READ X
ADD 100	ADD 100		READ Y		READ Y
WRITE Y	WRITE Y		DISPLAY X+Y		DISPLAY X+Y

Figure 5: Serial Schedule of two transactions

Schedule C: An Interleaved Schedule		
Schedule	T1	T2
READ X	READ X	
SUBTRACT 100	SUBTRACT 100	
READ X		READ X
WRITE X	WRITE X	
READ Y		READ Y
READ Y	READ Y	
ADD 100	ADD 100	
DISPLAY X+Y		DISPLAY X+Y
WRITE Y	WRITE Y	

Figure 6 (a): An Interleaved Schedule

Schedule D: A non-conflict equivalent schedule of Schedule C		
Schedule	T1	T2
READ X	READ X	
SUBTRACT 100	SUBTRACT 100	
WRITE X	WRITE X	
READ X		READ X
READ Y		READ Y
READ Y	READ Y	
ADD 100	ADD 100	
DISPLAY X+Y		DISPLAY X+Y
WRITE Y	WRITE Y	

Figure 6 (b): An Interleaved Schedule

You may observe that the Schedule C in Figure 6 is an interleaved schedule. It has conflicting operations, like reading and writing X and Y. There can be many interleaved schedules of transactions T1 and T2. How do you identify which two schedules are equivalent? Well, for that, let us define the term Conflict Equivalence.

Conflict Equivalence: Two schedules are defined as conflict equivalent if any two conflicting operations in the two schedules are in the same order. For example, Schedule D, as given below, is not a conflict equivalent schedule of Schedule C, as the sequence of WRITE X by T1 and READ X by T2 are not in the same order in the two interleaved schedules.

Conflict Serialisability or Serialisability

A schedule is called conflict serialisable if it is conflict equivalent to a serial schedule. In other words, for a given interleaved schedule, if the sequence of read and write is in the same order as that of any one of the serial schedules, then the interleaved schedule is called a conflict serialisable or serialisable schedule. In case an interleaved schedule is not serialisable, then it may result in problems due to concurrent execution of transactions.

Any schedule that produces the same results as a serial schedule is called a serialisable schedule. The basis of serialisability is taken from the notion of a serial schedule. Considering there are two concurrent transactions, T1 and T2, how many different serial schedules are possible for these two transactions? The possible serial schedules for two transactions, T1 and T2, are:

T1 followed by T2 OR T2 followed by T1.

Likewise, the number of possible serial schedules with three concurrent transactions would be defined by the number of possible permutations, which are:

T1-T2-T3	T1-T3-T2	T2-T1-T3
T2-T3-T1	T3-T1-T2	T3-T2-T1

But how can a schedule be determined to be serialisable or not? Is there any algorithmic way of determining whether a schedule is serialisable or not?

Using the notion of precedence graph, an algorithm can be devised to determine whether an interleaved schedule is serialisable or not. In this graph, the transactions of the schedule are represented as the nodes. This graph also has directed edges. An edge from the node representing transactions T_i to node T_j means that there exists a **conflicting operation** between T_i and T_j and T_i precedes T_j in some conflicting operations. The basic principle for determining a serialisable schedule is that the precedence graph, which determines the sequences of occurrence of transactions, does not contain a cycle. Given a precedence graph with no cycles, then it must be equivalent to a serial schedule. The steps of constructing a precedence graph are:

1. Create a node for every transaction in the schedule.
2. Find the precedence relationships in conflicting operations. Conflicting operations are (read-write) or (write-read) or (write-write) on the same data item in two different transactions. But how to find them?
 - 2.1 For a transaction T_i , which *reads* an item A , find a transaction T_j that *writes* A later in the schedule. If such a transaction is found, draw an edge from T_i to T_j .
 - 2.2 For a transaction T_i , which has *written* an item A , find a transaction T_j later in the schedule that *reads* A . If such a transaction is found, draw an edge from T_i to T_j .
 - 2.3 For a transaction T_i which has *written* an item A , find a transaction T_j that *writes* A later than T_i . If such a transaction is found, draw an edge from T_i to T_j .
3. If there is any cycle in the graph, the schedule is not serialisable, otherwise, find the equivalent serial schedule of the transaction by traversing the transaction nodes starting with the node that has no input edge.

Let us explain the algorithm with the help of the following example.

Example:

Let us use this algorithm to check whether the schedule given in Figure 6(a) is Serialisable. Figure 7 shows the required graph. Please note as per step 1, we draw the two nodes for T_1 and T_2 . In the schedule given in Figure 6, please note that the transaction T_2 reads data item X , which is subsequently written by T_1 , thus there is an edge from T_2 to T_1 (clause 2.1). Also, T_2 reads data item Y , which is subsequently written by T_1 , thus there is an edge from T_2 to T_1 (clause 2.1). However, that edge already exists, so we do not need to redo it. Please note that there are no cycles in the graph, thus, the schedule given in Figure 6 is **serialisable**. The equivalent serial schedule (as per step 3) would be T_2 followed by T_1 , which is serial Schedule-A of Figure 5.

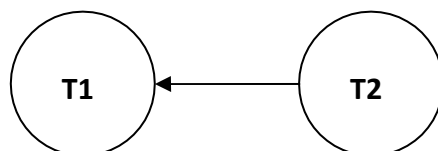


Figure 7: Test of Serialisability for the Schedule of Figure 6(a)

Please note that the schedule given in Figure 6(b) is not serialisable, because in that schedule, the two edges that exist between nodes T_1 and T_2 are:

- T_1 writes X which is later read by T_2 (clause 2.2), so there exists an edge from T_1 to T_2 .
- T_2 reads Y which is later written by T_1 (clause 2.1), so there exists an edge

from T2 to T1.

Thus, the graph for the schedule will be:

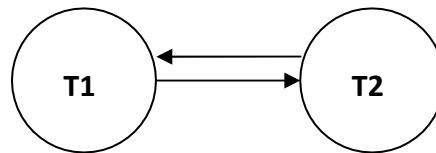


Figure 8: Test of Serialisability for the Schedule (c) of section 2.3

Please note that the graph above has a cycle T1-T2-T1, therefore it is **not serialisable**.

10.4.2 Locks

Serialisability is just a test of whether a given interleaved schedule is **serialisable** or has a concurrency related problem. However, it does not ensure that the interleaved concurrent transactions do not have any concurrency related problems. This can be done by using locks. So let us discuss what the different types of locks are, and then how locking ensures serialisability of executing transactions.

Types of Locks

There are two basic types of locks:

- Binary lock: This locking mechanism has two states for a data item: locked or unlocked.
- Multiple-mode locks: In this locking type each data item can be in three states read locked or shared locked, write locked or exclusive locked or unlocked.

Let us first take an example for binary locking and explain how it solves the concurrency related problems. Let us reconsider the transactions T1 and T2 and schedule given in Figure 4 (c) for this purpose; however, we will add required binary locks to them.

Schedule	T1	T2
LOCK X	LOCK X	
READ X	READ X	
SUBTRACT 100	SUBTRACT 100	
WRITE X	WRITE X	
UNLOCK X	UNLOCK X	
LOCK X		LOCK X
LOCK Y		LOCK Y
READ X		READ X
READ Y		READ Y
DISPLAY X+Y		DISPLAY X+Y
UNLOCK X		UNLOCK X
UNLOCK Y		UNLOCK Y
LOCK Y	LOCK Y	
READ Y	READ Y	
ADD 100	ADD 100	
WRITE Y	WRITE Y	
UNLOCK Y	UNLOCK Y	

Figure 9: An incorrect locking implementation

Does the locking as done above solve the problem of concurrent transactions? No, the same problems remain. Try working with the old values given in figure 4(c). Thus, locking should be done with some logic to make sure that locking results in no

concurrency related problem. One such solution is given below:

Schedule	T1	T2
LOCK X	LOCK X	
LOCK Y	LOCK Y	
READ X	READ X	
SUBTRACT 100	SUBTRACT 100	
WRITE X	WRITE X	
LOCK X (ISSUED BY T2)	LOCK X: denied as T1 holds the lock. The transaction T2 Waits and T1 continues.	
READ Y	READ Y	
ADD 100	ADD 100	
WRITE Y	WRITE Y	
UNLOCK X	UNLOCK X	
	LOCK X request of T2 on X will be granted and transaction T2 resumes.	
UNLOCK Y	UNLOCK Y	
LOCK Y		LOCK Y
READ X		READ X
READ Y		READ Y
DISPLAY X+Y		DISPLAY X+Y
UNLOCK X		UNLOCK X
UNLOCK Y		UNLOCK Y

Figure 10: A correct but restrictive locking implementation.

As shown in Figure 10, when you obtain all the locks at the beginning of the transaction and release them at the end, it ensures that transactions are executed with no concurrency-related problems. However, such a scheme limits the concurrency. We will discuss a two-phase locking method in the next subsection that provides sufficient concurrency. However, let us first discuss multiple-mode locks.

Multiple-mode locks: It offers two locks: shared locks and exclusive locks. But why do we need these two locks? There are many transactions in the database system that never update the data values. These transactions can coexist with other transactions that update the database. In such a situation multiple reads are allowed on a data item, so multiple transactions can lock a data item in the shared or read lock. On the other hand, if a transaction is an updating transaction, that is, it updates the data items, it must ensure that no other transaction can access (read or write) those data items that it wants to update. In this case, the transaction places an exclusive lock on the data items. Thus, a higher level of concurrency can be achieved compared to the binary locking scheme. The properties of shared and exclusive locks are summarised below:

a) Shared lock

- It is requested by a transaction that wants to just read the value of the data item.
- A shared lock on a data item does not allow an exclusive lock to be placed but permits any number of shared locks to be placed on that item.

b) Exclusive lock

- It is requested by a transaction on a data item that it needs to update.
- No other transaction can place either a shared lock or an exclusive lock on a data item that has been locked in an exclusive mode.

We explain these locks with the help of an example. However, you may refer to further readings for more information on multiple-mode locking. We will once again consider

the transactions T1 and T2, but in addition, a transaction T11 that finds the total of accounts Y and Z.

Schedule	T1	T2	T11
S_LOCK X		S_LOCK X	
S_LOCK Y		S_LOCK Y	
READ X		READ X	
S_LOCK Y			S_LOCK Y
S_LOCK Z			S_LOCK Z
			READ Y
			READ Z
X_LOCK X	X_LOCK X. The exclusive lock request on X is denied as T2 holds the Shared lock. The transaction T1 Waits.		
READ Y		READ Y	
DISPLAY X+Y		DISPLAY X+Y	
UNLOCK X		UNLOCK X	
X_LOCK Y	X_LOCK Y. The previous exclusive lock request on X is granted as X is unlocked. But the new exclusive lock request on Y is not granted as Y is locked by T2 and T11 in Shared mode. Thus, T1 waits till both T2 and T11 will release the Shared lock on Y.		
DISPLAY Y+Z			DISPLAY Y+Z
UNLOCK Y		UNLOCK Y	
UNLOCK Y			UNLOCK Y
UNLOCK Z			UNLOCK Z
READ X	READ X		
SUBTRACT 100	SUBTRACT 100		
WRITE X	WRITE X		
READ Y	READ Y		
ADD 100	ADD 100		
WRITE Y	WRITE Y		
UNLOCK X	UNLOCK X		
UNLOCK Y	UNLOCK Y		

Figure 11: Example of Locking in multiple modes

As shown in Figure 11, locking can result in a serialisable schedule. Next, we discuss the concept of granularity of locking.

Granularity of Locking

In general, locks may be allowed on the following database items:

- 1) The complete database itself
- 2) A file of the database
- 3) A page or disk block of data
- 4) A record in a table
- 5) A data item of an attribute

Granularity of locking depends on the size of database item being locked. A coarse granularity locking means locking a larger data item, e.g. the complete database or file or page etc. The fine granularity locking is locking the database items of the smaller size, e.g. the record locking or the data item locking.

A lock can be implemented as a binary variable, which requires space. In addition, a lock requires locking and unlocking operations. Thus, coarse locking has low locking overheads, but it restricts the concurrency among transactions. For example, if a single transaction locks the complete database in an exclusive mode, all other transactions will wait for the completion of this transaction. On the other hand, the fine granularity of locking results in high locking overheads.

To make an efficient concurrent transaction processing system, multiple granularity locking is supported by a database system. These system support Intention mode locking. You may refer to the further readings for more details on such locking scheme.

In the next section, we discuss the locking protocol that ensures correct execution of concurrent transactions.

10.4.3 Two-Phase Locking (2PL)

In this section we try to answer the question: Can you release locks a bit early and still have no concurrency related problem? Yes, we can do it if we use two-phase locking protocol. The two-phase locking protocol consists of two phases:

Phase 1: The lock acquisition phase: If a transaction T wants to read an object, it needs to obtain the S (shared) lock. If transaction T wants to modify an object, it needs to obtain X (exclusive) lock. No conflicting locks are granted to a transaction. **New locks on items can be acquired but no lock can be released, till all the locks required by the transaction are obtained.**

Phase 2: Lock Release Phase: The existing locks can be released in any order, but no new lock can be acquired **after a lock has been released**. The locks are held only till they are required.

Normally, any legal schedule of transactions that follows two-phase locking protocol is guaranteed to be serialisable. The two-phase locking protocol has been proven for its correctness. However, the proof of this protocol is beyond the scope of this Unit. You can refer to further readings for more details on this protocol.

There are two types of 2PL:

- (1) Conservative 2PL
- (2) Strict 2PL

The conservative 2PL allows the release of the lock at any time after all the locks have been acquired. For example, you can release the locks in the schedule of *Figure 10*, after you have read the values of Y and Z in transaction 11, even before the display of the sum. This will enhance the concurrency level. The conservative 2PL is shown graphically in *Figure 12*.

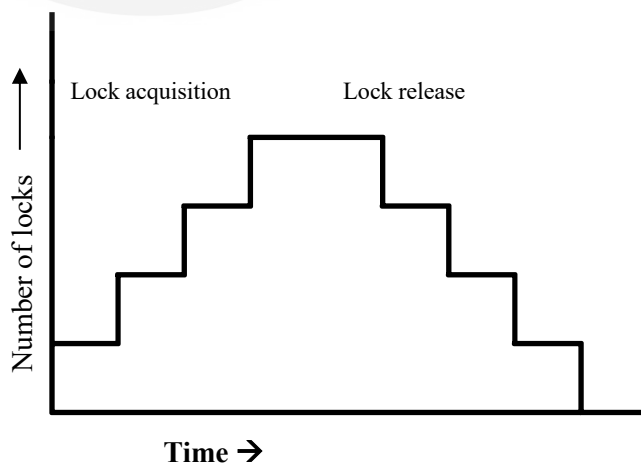


Figure 12: Conservative Two-Phase Locking

However, conservative 2PL suffers from the problem that it can result in loss of atomic or isolation property of transaction as theoretically speaking, once a lock is released on

a data item, it can be modified by another transaction before the first transaction commits or aborts.

To avoid such a situation, you use strict 2PL. The strict 2PL is graphically depicted in *Figure 13*. However, the basic disadvantage of strict 2PL is that it restricts concurrency as it locks the item beyond the time it is needed by a transaction.

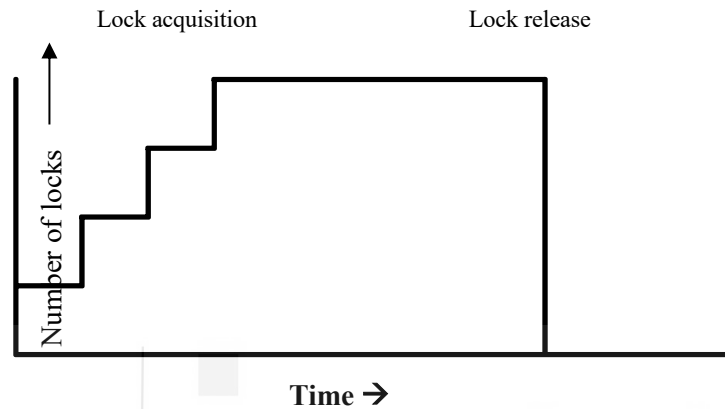


Figure 13: Strict Two-Phase Locking

Does the 2PL solve all the problems of concurrent transactions? No, the strict 2PL solves the problem of concurrency and atomicity; however, it introduces another problem: “Deadlock”. Let us discuss this problem in the next section.

☞ Check Your Progress 2

- 1) Let the transactions T1, T2, T3 perform the following operations:

T1: Add one to A

T2: Double A

T3: Display A on the screen and set A to one.

Suppose transactions T1, T2, and T3 are allowed to execute concurrently. If A has an initial value of zero, how many possible correct results are there? Enumerate them.

.....

- 2) Consider the following two transactions on two bank accounts having a balance A and B.

Transaction T1: Transfer Rs. 100 from A to B

Transaction T2: Find the multiple of A and B.

Create a non-serialisable schedule.

.....

- 3) Add lock and unlock instructions (exclusive or shared) to transactions T1 and T2 so that they observe the serialisable schedule. Make a valid schedule.

10.5 DEADLOCK HANDLING AND ITS PREVENTION

As seen earlier, though the 2PL protocol handles the problem of serialisability, but it causes some problems also. For example, consider the following two transactions and a schedule involving these transactions:

TA	TB
X_LOCK A	X_LOCK A
X_LOCK B	X_LOCK B
:	:
:	:
UNLOCK A	UNLOCK A
UNLOCK B	UNLOCK B

Schedule

```

T1: X_LOCK A
T2: X_LOCK B
T1: X_LOCK B
T2: X_LOCK A
  
```

As is clearly seen, the schedule causes a problem. After T1 has locked A, T2 locks B and then T1 tries to lock B, but is unable to do so and waits for T2 to unlock B. Similarly, T2 tries to lock A but finds that it is held by T1 which has not yet unlocked it and thus waits for T1 to unlock A. At this stage, neither T1 nor T2 can proceed since both transactions are waiting for the other to unlock the locked resource (refer to figure 14).

Clearly, the schedule comes to a halt in its execution. The important thing to be seen here is that both TA and TB follow the 2PL, which guarantees serialisability. Whenever the situation, i.e. all the transactions are **waiting for a condition that will never occur**, arises, we say that a deadlock has occurred.

The deadlock can be described in terms of a directed graph called a “wait for” graph, which is maintained by the lock manager of the DBMS. This graph G is defined by the pair (V, E). It consists of a set of vertices/nodes V and a set of edges/arcs E. Each transaction is represented by node and an arc from $T_i \rightarrow T_j$, if T_j holds a lock on data items that T_i is waiting for. When transaction T_i requests a data item currently being held by transaction T_j then the edge $T_i \rightarrow T_j$ is inserted in the “wait for” graph. This edge is removed only when transaction T_j is no longer holding the data item needed by transaction T_i .

A deadlock in the system of transactions occurs, if and only if the wait-for graph contains a cycle. Each transaction involved in the cycle is said to be deadlocked. To detect deadlocks, a periodic check for cycles in “wait-for” graph can be done. For example, the “wait-for” for the schedule of transactions TA and TB, as given earlier in this section, is shown in Figure 14.

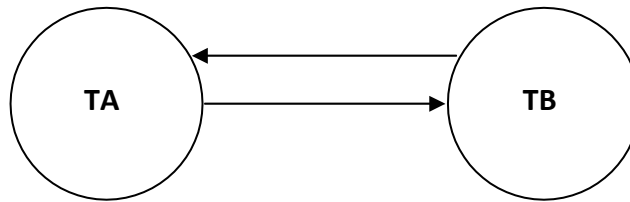


Figure 14: Wait For Graph of TA and TB

In Figure 14, TA and TB are the two transactions. The two edges are present between nodes TA and TB since each is waiting for the other to unlock a resource held by the other, forming a cycle and causing a deadlock problem. The above case shows a direct cycle. However, in actual situations, more than two nodes may be there in a cycle.

A deadlock is a situation that can be created because of locks. It causes transactions to wait forever hence the name deadlock. A deadlock occurs because of the following conditions:

- Mutual exclusion: A resource can be locked in exclusive mode by only one transaction at a time.
- Non-preemptive locking: A data item can only be unlocked by the transaction that locked it. No other transaction can unlock it.
- Partial allocation: A transaction can acquire locks on a database in a piecemeal way.
- Circular waiting: Transactions lock part of the data resources needed and then wait indefinitely to lock the resource currently locked by other transactions.

In order to prevent a deadlock, one has to ensure that at least one of these conditions does not occur.

A deadlock can be prevented, avoided, or controlled. Let us discuss a simple method for deadlock prevention. You can refer to other methods from the further readings.

Deadlock Prevention

One of the simplest approaches for avoiding a deadlock would be to acquire all the locks at the start of the transaction. However, this approach restricts concurrency greatly, also you may lock some of the items that are not updated by that transaction. A deadlock prevention algorithm prevents a deadlock. It uses the approach: *do not allow circular wait*. This approach rolls back some of the transactions instead of letting them wait.

There exist two such schemes. These are:

“Wait-die” scheme: The scheme is based on non-preventive technique. It is based on a simple rule:

If T_i requests a database resource that is held by T_j
 then if T_i has a smaller timestamp than that of T_j
 it is allowed to wait;
 else T_i aborts.

A timestamp may loosely be defined as the system-generated sequence number that is unique for each transaction. Thus, a smaller timestamp means an older transaction. For example, assume that three transactions T_1 , T_2 and T_3 were generated in that sequence; then if T_1 requests for a data item which is currently held by transaction T_2 ,

it is allowed to wait as it has a smaller time stamping than that of T1. However, if T3 requests for a data item that is currently held by transaction T2, then T3 is rolled back (die).

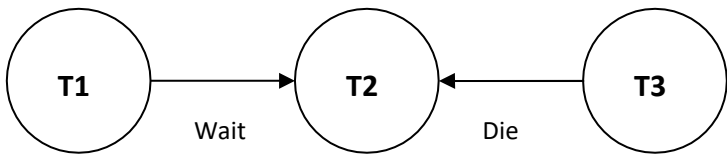


Figure 15: Wait-die Scheme of Deadlock Prevention.

“Wound-wait” scheme: It is based on a preemptive technique and is explained with the help of the following sequence of actions:

Resource Requesting Transaction	Resource is with the Transaction	Check Timestamp (TSP)	Action Needed
Ti	Tj	TSP (Ti) > TSP (Tj)	Ti is younger, therefore, can WAIT
		TSP (Ti) < TSP (Tj)	Ti is older, therefore, WOUND Ti

For example, consider three transactions T1, T2 and T3 with $TSP(T1) < TSP(T2) < TSP(T3)$, i.e. T1 is the oldest and T3 is the youngest. For example, consider the following sequence of requests for a data item X:

- T2 requests for X Action: Locks the Resource
- T1 requests for X Action: Wound (Rollback) T1, as it is older.
- T3 requests for X Action: T3 Waits for T2 to release X, as it is younger.

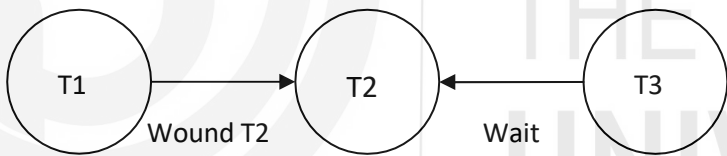


Figure 15: Wound-wait Scheme of Deadlock Prevention.

It is important to check that no transaction should get rolled back repeatedly such that it is never allowed to make progress. This is referred to starvation of a transaction. Also, both “wait-die” and “wound-wait” scheme should avoid starvation. The number of aborts and rollbacks will be higher in wait-die scheme than in the wound-wait scheme. But one major problem with both schemes is that these schemes may result in unnecessary rollbacks. You can refer to further readings for more details on deadlock-related schemes.

10.6 OPTIMISTIC CONCURRENCY CONTROL

Is locking the only way to prevent concurrency-related problems? There exist some other methods too. One such method is called an Optimistic Concurrency control. Let us discuss it in more detail in this section.

In general, the concurrency-related problem does not occur frequently, therefore, an

optimistic concurrency control scheme allows database transactions to freely perform the update operations on data items. Further, in the validation phase a transaction cross-checks if any other transaction has modified the data items, which it had read or written. Therefore, the optimistic concurrency control algorithm has the following phases:

- a) **READ Phase:** In this phase, a transaction makes a local copy of the data items needed by it in its own memory space. The transaction then makes changes in the local copies of data items.
- b) **VALIDATE Phase:** In this phase, the transaction validates the values of data items read by the transaction during the READ phase. This validation ensures that no other transaction has changed the values of the data items, while this transaction was modifying the local copy of the data items. In case, the validation is unsuccessful, then this transaction is aborted, and the local updates of data items are discarded. However, in case of successful validation, the Write phase is performed.
- c) **WRITE Phase:** This phase is performed if the Validate Phase is successful. In this phase, the transaction commits and all the data item updates made by the transaction in the local copies, are applied to the database.

To explain the optimistic concurrency control, the following terms are used:

- **Write set (WS(T)):** Given a transaction T, WS(T) is the set of data items that would be written by it.
- **Read set (RS(T)):** Given a transaction T, RS(T) is the set of data items that would be read by it.

More details on this scheme are available in further readings. But let us show this scheme here with the help of the following examples: Consider the set for transactions T1 and T2.

T1		T2	
Phase	Operation	Phase	Operation
-	-	Read	Reads the RS(T2), say variables X and Y and performs updating of local values
Read	Reads the RS(T1), say variable A and B and performs updating of local values	-	-
Validate	Validate the values of RS(T1)	-	-
-	-	Validate	Validate the values of RS(T2)
Write	Transaction Commits and writes the updated values WS(T1) in the database.	-	-
-	-	Write	Transaction Commits and writes the updated values WS(T2) in the database.

In this example, both T1 and T2 commit. Please note that read sets RS(T1) and RS(T2) are disjoint, also the Write sets are also disjoint, thus, no concurrency-related problem occurs.

Now let us consider another example, as given below:

T1	T2	T3
Read A	--	--
--	Read A	--
--	--	Read D
--	--	Update D
--	--	Update A
--	--	Validate (D, A) finds OK
--	--	Write (D, A), COMMIT
--	Validate (A): Unsuccessful Value changed by T3	--
Validate (A): Unsuccessful Value changed by T3	--	--
ABORT T1	--	--
--	ABORT T2	--

In this case, both T1 and T2 get aborted as they fail during validate phase while only T3 is committed. Optimistic concurrency control performs its checking at the transaction commit point in a validation phase. The serialisation order is determined by the time of the transaction validation phase.

10.7 TIMESTAMP BASED PROTOCOLS

A timestamp is used to identify the sequence of transactions. This section explains the timestamp-based concurrent execution of transactions and the multi-version technique.

10.7.1 Timestamp-Based Concurrency Control

Timestamp-based protocols use the concept of a timestamp, which is a unique value assigned to a transaction that can determine the sequence or order of transactions. The timestamp of a transaction can either be the clock time at the start of a transaction or it can be an auto-incrementing counter. In addition, the timestamping-based protocol requires that each data item be associated with the following timestamps:

1. Read timestamp: Read timestamp of a data item, say X, is the highest timestamp of the transaction that has read that data item.
2. Write timestamp: Write timestamp of a data item is the highest timestamp of the transaction that has written that data item.

Assume that a data item D_i has a read timestamp rDS_i and write timestamp wDS_i ; and a transaction with timestamp $trSi$ makes a request to READ D_i , then:

READ operation on D_i :

IF ($trSi < wDS_i$) then REJECT READ and ROLLBACK T_i

//Why? The transaction is trying to read the data item, which has a lower timestamp than that of the highest timestamp of the transaction that has written the data item. Thus, this transaction is too late to read the data item, which is already written by a younger transaction. //

ELSE // $trSi \geq wDS_i$ is TRUE//

so ALLOW READ and

SET rDS_i = higher of ($trSi$, rDS_i) //Set read timestamp of data item

Write operation on Di:

IF ($trSi < rDSi$) then REJECT WRITE and ROLLBACK T_i

//Why? The transaction is trying to write the data item, which is already read by a newer transaction. Therefore, this transaction is too late to write the data item.

IF ($trSi < wDSi$) then REJECT WRITE and ROLLBACK T_i

Why? The transaction is trying to write the data item, which is already written by a newer transaction. Therefore, this transaction is trying to write an old value to the data item.

ELSE // $trSi \geq rDSi$ and $trSi \geq wDSi$ //

so ALLOW WRITE and

SET $wDSi = trSi$

For example, consider the transactions T_1 with timestamp 1, T_2 with timestamp 2 and T_3 with timestamp 3. A data item D_1 is read by these transactions in the sequence T_2 , T_1 and T_3 . In addition, they request to write the data item D_1 in the sequence T_3 , T_1 , T_2 . Then how are these sequences allowed or rejected? The following Figure shows this sequence:

Request	Timestamp of the Transaction ($trSi$)	Action Performed	Read Timestamp of data item D_1 ($rDSi$)	Write Timestamp of data item D_1 ($wDSi$)
Initial State	-	-	0	0
READ D_1 by T_2	2	Allowed	2	0
READ D_1 by T_1	1	$trS1 < rDS1$ Reject and Rollback T_1	2	0
READ D_1 by T_3	3	$trS3 > rDS1$ Allowed	3	0
WRITE D_1 by T_3	3	$trS3 \geq rDS1$ and $trS3 > wDS1$ Allowed	3	3
WRITE D_1 by T_1	-	This Transaction is already Rolled back	3	3
WRITE D_1 by T_2	2	$trS2 < rDS1$ Reject and Rollback T_2	3	3

Thus, you can observe that most of the rules have been applied in the example, as given above.

10.7.2 Multi-version Technique

Multi-version technique, as the name suggests, allows the previous versions of data to be stored. Thus, this scheme avoids the rollback of transactions. This scheme also uses read timestamp and write timestamp for each version of the data item. This technique can be defined as:

Consider a data item D_i and its version $D_i(Ver1)$, $D_i(Ver2)$, ..., $D_i(Vern)$. For each $D_i(Veri)$, a read timestamp $rDSi(Veri)$ and a write timestamp $wDSi(Veri)$ are stored. The read timestamp for the multi-version technique is the highest timestamp of the

transactions that have read the i^{th} version; whereas the write timestamp is the timestamp of the transaction that has resulted in the creation of this version.

In this technique read operation reads the version whose timestamp is equal to or just less than the transaction. While the case of a write operation by a transaction with timestamp $trSi$, which is trying to write a data item Di that has k versions, the following three cases are possible:

- IF $trSi < rDSi(Verk)$, then ROLLBACK transaction;
- IF $trSi = wDSi(Verk)$, then REWRITE the version k of the data item;
- IF $trSi > rDSi(Verk)$, then CREATE version $k+1$ of data item;

10.8 WEAK LEVEL OF CONSISTENCY AND SQL COMMANDS FOR TRANSACTIONS

In general, concurrent transactions are expected to be operated in serialisable mode of operation. However, in many applications, certain weaker consistency levels can also be used. These weaker consistency level puts lesser restrictions on the execution of transactions. In this section, we discuss the four different isolation levels supported by SQL, a few of which supports a weak level of consistency.

SERIALISABLE: In this isolation level, the transactions follow the ACID properties, therefore, eliminating problems due to concurrent execution of transactions, however, restricting the concurrency.

REPEATABLE READ: This isolation level allows the reading of data items only after the related transaction has been committed. However, in this isolation level a recently committed transaction may perform addition or removal of some rows.

READ COMMITTED: This isolation level also allows the reading of data items only after the related transaction has been committed. However, if a transaction repeats a read operation of a specific data item, then it is not guaranteed to obtain the same value for that data item.

READ UNCOMMITTED: This is the weakest isolation level and does not put any restriction on the concurrent execution of transactions. However, it may result in every concurrency-related problem.

The SQL command that supports transactions is:

To set a typical isolation level for transaction execution, you may set the transaction isolation level using the command:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

You may also change the isolation level by using the alter command. You can commit a transaction by using COMMIT or roll it back by using ROLLBACK in the transaction.

You can refer to further readings for a detailed discussion on weak consistency levels and SQL commands related to transaction control.



Check Your Progress 3

- 1) Draw a suitable graph for following locking requests and find whether the transactions are deadlocked or not.

T1: S_LOCK A	--	--
--	T2: X_LOCK B	--
--	T2: S_LOCK A	--
--	--	T3: X_LOCK C
--	T2: S_LOCK C	--
T1: S_LOCK B	--	--
T1: S_LOCK A	--	--
--	--	T3: S_LOCK A
All the unlocking requests start from here		

.....

.....

.....

2) What is Optimistic Concurrency Control?

.....

.....

3) What are the different types of timestamps for a data item? Why are they used?

.....

.....

4) What is the purpose of the multi-version technique? Does the multi-version technique also have rollback?

.....

.....

5) What are the different weak consistency levels?

.....

.....

10.9 SUMMARY

In this unit you have gone through the concepts of transaction and Concurrency Management. A transaction is a sequence of many actions. Concurrency control deals with ensuring that two or more transactions do not get into each other's way, i.e., updates of data items made by one transaction do not affect the updates made by the other transactions on the same data items.

Serialisability is the generally accepted criterion for the correctness of concurrency control. It is a concept related to concurrent schedules. It determines whether any schedule is serialisable or not. Any schedule that produces the same results as a serial schedule is a serialisable schedule.

Concurrency Control is usually done via locking. If a transaction tries to lock a resource already locked by some other transaction, it cannot do so and waits for the source to be

unlocked. This unit also presents the Two-Phase Locking (2PL) protocol that ensures that transactions do not encounter concurrency-related problems. A system is in a deadlock state if there exists a set of transactions, which are waiting for each other to complete. We can use a deadlock prevention protocol to ensure that the system will never enter a deadlock state.

This unit also discusses optimistic concurrency control, timestamp-based concurrency control and multi-version technique for concurrency control. Finally, the unit discusses the different weak consistency levels and related SQL commands.

10.10 SOLUTIONS/ANSWERS

Check Your Progress 1

- 1) A transaction is the basic unit of work on a Database management system. It defines the data processing operations on the database. It has four basic properties:
- a) Atomicity: transaction is done completely or not at all.
 - b) Consistency: Leaves the database in a consistent state
 - c) Isolation: Other transactions should not see uncommitted values
 - d) Durability: Once committed the changes should be reflected in the database.

A transaction can update more than one data values. Some transactions can do writing of data without reading a data value.

A simple transaction example may be: Updating the stock inventory of an item that has been issued.

- 2) The basic anomalies of concurrent transactions are:
- Lost update anomaly: An update is overwritten.
 - Unrepeatable read anomaly: On reading a value later again an inconsistent value is found.
 - Dirty read anomaly: Reading an uncommitted value.
 - Inconsistent analysis: Due to reading partially updated value.

No, these problems cannot occur if the transactions do not perform interleaved read or write operations on the same data items.

- 3) The commit state arrives when a transaction has completed all the updates correctly, and all these changes are to be accepted. No, you cannot roll back after the commit.

Check Your Progress 2

- 1) There are six possible results, corresponding to six possible serial schedules:

Initially:	Sequence of operation as initially A=0	Final Value
T1-T2-T3:	A = 1; A=2; A is displayed as 2 and Set to 1	1
T1-T3-T2:	A = 1; A is displayed as 1 and set to 1; A=2	2
T2-T1-T3:	A = 0; A=1; A is displayed as 1 & set to 1	1

T2-T3-T1:	A = 0; A is displayed as 0 and set to 1; A=2	2
T3-T1-T2:	A is displayed as 0 and set to 1; A=2; A=4	4
T3-T2-T1:	A is displayed as 0 and set to 1; A=2; A=3	3

2)

Schedule	T1	T2
READ A	READ A	
A = A - 100	A = A - 100	
WRITE A	WRITE A	
READ A		READ A
READ B		READ B
READ B	READ B	
RESULT = A * B		RESULT = A * B
DISPLAY RESULT		DISPLAY RESULT
B = B + 100	B = B + 100	
WRITE B	WRITE B	

Please make the precedence graph and find out that the schedule is not serialisable.

3) This is a schedule using conservative 2 PL.

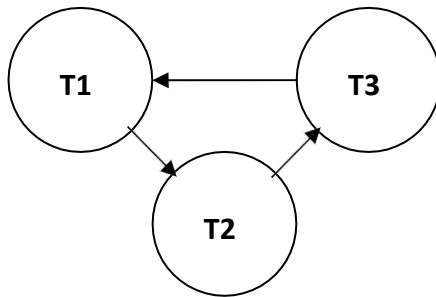
Schedule	T1	T2
Lock A	Lock A	
Lock B	Lock B	
Read A	Read A	
A = A - 100	A = A - 100	
Write A	Write A	
Unlock A	Unlock A	
Lock A		Lock A: Granted
Lock B		Lock B: Waits
Read B	Read B	
B = B + 100	B = B + 100	
Write B	Write B	
Unlock B	Unlock B	
Read A		Read A
Read B		Read B
Result = A * B		Result = A * B
Display Result		Display Result
Unlock A		Unlock A
Unlock B		Unlock B

You must also make the schedules using read and exclusive lock and a schedule in strict 2PL.

Check Your Progress 3

- 1) Transaction T1 gets the shared lock on A, T2 gets the exclusive lock on B and Shared lock on A, while transaction T3 gets the exclusive lock on C.
 - Now T2 requests for a shared lock on C, which is exclusively locked by T3, so this lock cannot be granted. So T2 waits for T3 on item C.
 - T1 now requests for Shared lock on B, which is exclusively locked by T2, thus, it waits for T2 for item B. T1 request for a shared lock on C is not processed.
 - Next, T3 requests for an exclusive lock on A, which is share locked by T1, so it cannot be granted. Thus, T3 waits for T1 for item A.

The Wait for graph for the transactions for the given schedule is:



Since there exists a cycle, therefore, the schedule is deadlocked.

- 2) The basic philosophy for optimistic concurrency control is the optimism that nothing will go wrong, so let the transaction interleave in any fashion, but to avoid any concurrency-related problem, you just validate your assumption before you make changes permanent. This is a good model for situations having a low rate of transactions.
- 3) Two basic timestamps are associated with a data item – Read timestamp and Write timestamp. They are primarily used to see that a transaction that is reading data should not be too old that some younger transaction has already written it, or a transaction that wants to write that data item should not be younger than the transaction that has read or written it.
- 4) The multi-version technique of concurrent transactions ensures that all the versions or changes in the value of a data item are duly recorded. Yes, even this scheme may have a rollback.
- 5) The four weak consistency levels are - SERIALISABLE, REPEATABLE READ, READ COMMITTED, READ UNCOMMITTED.